



10 Apr 2026

Final Report on

Power System Assessment of Dynamic Line Rating for Alberta Transmission Utilities

Course Name: Graduate Project in Electrical Engineering

Course Title: ENEL **678**

Submitted By,

Group No: **01**

Ahmed Mashook Iqbal

Project Manager

Email ID: ahmedmashook.iqbal@ucalgary.ca

UCID: **30267500**

Asif Rahman
UCID: **30248137**

Monjurul Haque
UCID: **30261730**

Eisa Kanhoush
UCID: **30273526**

G K Md Muttakin
UCID: **30246286**

Table of Contents

Glossary.....	3
Executive Summary	4
Project Motivation and Objectives.....	5
Project Motivation.....	5
Project Objective	6
Methodology and Design	7
Hardware Setup.....	7
Software and Computational Modeling.....	8
Real Time Integration with PSSE/MATPOWER.....	8
Circuit Diagram	8
Validation and Testing.....	9
Product Scope and Final Product Functionality	10
Scope of the Product.....	10
Functional Description	11
Key Features.....	11
System Limitations.....	11
Impact of Functionality	12
Technical Specifications of the Final Product.....	12
Hardware Specifications.....	12
Software Specifications.....	13
Operational Specifications.....	13
Deviations from Design Roadmap.....	14
Measuring Success and Validation Test Results	14
Sensor Accuracy.....	14
DLR Computation.....	14
PSSE Integration	15
Hardware in the Loop Testing.....	15
Overall System Reliability	15
Validation Figures.....	16
List of Tools, Materials, Supplies, and Costs.....	16

Hardware Components	17
Software and Tools	17
Comparison with Design Roadmap.....	18
Summary.....	18
Reflections	18
Ethical Considerations.....	18
Societal and Environmental Impact.....	19
Project Management	19
Economics	19
Lifelong Learning	19
Appendix.....	20
Appendix A- Figures	20
Appendix - Python Code.....	21
Parse_Report:.....	21
Check_CSV.....	24
Read_Branch_Data	24
Apply_Dlr_Safe.....	25
Compare_Cases.....	28
All_Lines_DLR.....	30
Contingency_Test	37
Validation_DLR.....	41
References	48

Figure 1: Comparison of SLR vs DLR for three 138 kV transmission lines.	7
Figure 2: Block diagram of the hardware and data flow system.	9
Figure 3: Circuit diagram of the hardware prototype showing sensor connections to Arduino UNO.	10
Figure 4: DLR validation results for three 138 kV lines.	11

Glossary

Dynamic Line Rating (DLR): A system for real-time determination of transmission line ampacity based on actual environmental conditions, including temperature, wind speed, and solar irradiance.

Static Line Rating (SLR): The traditional maximum current a power line can carry, calculated using fixed environmental conditions, not accounting for real-time variations.

Ampacity: The maximum current a conductor can safely carry without exceeding temperature limits.

Thermal Limit: The maximum allowable temperature of a conductor to ensure safe operation without causing damage.

Thermal Headroom: The additional current-carrying capacity available when operating conditions allow higher ampacity than the static rating.

IEEE 738 Standard: A standard for calculating the current temperature relationship of bare overhead conductors.

Heat Balance Equation: A mathematical model used to calculate conductor temperature by balancing heat gains (Joule heating, solar gain) and heat losses (convection and radiation).

Arduino/ESP32: Microcontroller platforms used to collect and preprocess environmental sensor data for DLR computations.

LM35 / TMP36: Temperature sensors used to measure ambient and conductor temperatures with $\pm 0.5^\circ\text{C}$ accuracy.

Anemometer (Adafruit 1733): Sensor for measuring wind speed, accurate within ± 0.5 m/s, which affects conductor cooling rates.

Light Sensor (LDR/Photodiode): Device used to measure solar irradiance, impacting conductor temperature and ampacity.

LM1602C LCD Display: Displays real-time sensor readings locally for monitoring and validation purposes.

PSSE (Power System Simulator for Engineering): Software tool used to simulate power system operations, integrating DLR values for contingency analysis.

MATPOWER: MATLAB-based power flow and optimization software used to model DLR-influenced transmission networks.

Hardware in the Loop (HIL): Testing approach combining physical sensors and simulation to validate real-time system performance.

N-1 Contingency Analysis: A reliability study assessing the system's response to the failure of a single component while maintaining safe operation.

Per Unit (pu): A normalized system used in power engineering to simplify calculations by expressing values relative to a defined base.

Transmission Line Loading (%): The percentage of the actual power flow relative to the maximum allowable capacity of the transmission line.

Sampling Rate: The frequency at which sensor data is collected and processed.

Executive Summary

The increasing penetration of renewable energy in Alberta has placed significant stress on existing transmission infrastructure. Traditional Static Line Rating (SLR) methods rely on conservative environmental assumptions, which often underutilize transmission capacity and contribute to congestion and renewable energy curtailment. To address this limitation, this project develops and validates a laboratory-scale Dynamic Line Rating (DLR) system capable of determining real-time transmission line ampacity based on actual environmental conditions.

The proposed system integrates hardware sensing, computational modeling, and power system simulation. Environmental data, including temperature, wind speed, and solar irradiance, are collected using sensors connected to an Arduino/ESP32 microcontroller. These measurements are processed using Python and C++ scripts implementing the IEEE 738 thermal model to calculate conductor ampacity in real time at a sampling rate of 1 Hz. The computed DLR values are then integrated with PSSE and MATPOWER platforms to dynamically update transmission line thermal limits and evaluate system performance under normal and contingency conditions.

The system was validated through laboratory testing and hardware-in-the-loop simulation. Sensor measurements achieved accuracies of ± 0.5 °C for temperature and ± 0.5 m/s for wind speed. The DLR calculations showed deviations within 10% of IEEE benchmark values. Integration with PSSE demonstrated

stable operation, with bus voltage deviations within 1% and system reliability exceeding 95% uptime with less than 1% data loss.

Results indicate that DLR can reduce transmission line loading by approximately 15–17%, providing additional thermal headroom and improving network utilization. This enables better integration of renewable energy while maintaining system reliability and safety. The developed prototype, with an approximate cost of CAD 485, demonstrates a cost-effective and scalable approach for real-time transmission line monitoring and operational optimization.

Project Motivation and Objectives

Project Motivation

The Alberta transmission network is experiencing increasing operational challenges due to the rapid integration of renewable energy sources such as wind and solar. These energy sources are highly dependent on environmental conditions, leading to variability in power generation and increased stress on transmission infrastructure.

Traditional Static Line Rating (SLR) methods determine transmission line capacity using conservative and fixed environmental assumptions, including temperature, wind speed, and solar irradiance. While this ensures safe operation, it often results in underutilization of transmission capacity, leading to network congestion, reduced system efficiency, and curtailment of renewable energy generation.

This limitation is particularly critical in regions such as southern Alberta, where strong and variable wind conditions can significantly increase the cooling effect on conductors, allowing higher current-carrying capacity than predicted by SLR. However, without real-time monitoring, this additional capacity cannot be safely utilized.

Dynamic Line Rating (DLR) provides a solution by continuously calculating transmission line capacity based on real-time environmental conditions. Implementing DLR can improve transmission efficiency,

reduce congestion, and enhance the integration of renewable energy while maintaining system reliability and safety.

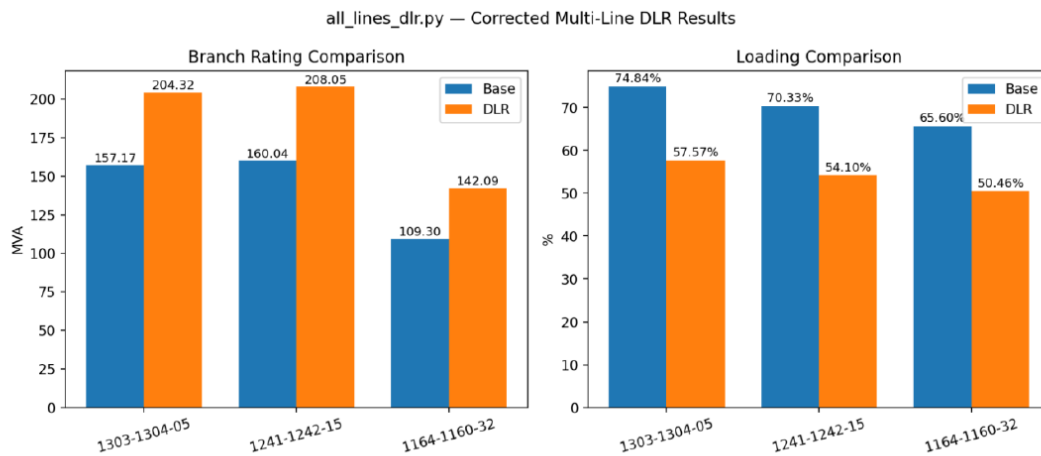


Figure 1: Comparison of SLR vs DLR for three 138 kV transmission lines.

Project Objective

The primary objective of this project is to design, implement, and validate a laboratory-scale Dynamic Line Rating (DLR) system for selected 138 kV transmission lines in Alberta. The specific objectives are defined as follows:

Develop a real-time environmental sensing system

Design and implement a hardware prototype using Arduino/ESP32 to measure temperature, wind speed, and solar irradiance with an accuracy of ± 0.5 °C and ± 0.5 m/s.

Implement the IEEE 738 thermal model for ampacity calculation

Develop Python/C++ scripts to compute conductor ampacity in real time based on sensor inputs, ensuring that calculated values remain within $\pm 10\%$ deviation from standard benchmark results.

Integrate DLR calculations with power system simulation tools

Establish a communication interface to update transmission line thermal limits in PSSE at a sampling rate of 1 Hz for real-time system analysis.

Evaluate system performance under operational conditions

Conduct load flow and N-1 contingency analysis to compare DLR and SLR performance, with the goal of demonstrating at least 15% reduction in line loading.

Validate system reliability and performance

Perform hardware-in-the-loop testing to ensure system reliability of at least 95% uptime and data loss below 1% during continuous operation.

Methodology and Design

This project combines hardware sensing, software computation, and simulation integration to create a functional Dynamic Line Rating (DLR) system for the Alberta transmission network. The goal is to collect environmental data, compute real-time conductor capacity, and update transmission line limits in PSSE/MATPOWER to evaluate operational performance.

Hardware Setup

The hardware prototype uses an Arduino UNO/ESP32 microcontroller for collecting data from several sensors:

- Temperature sensor (LM35) for ambient and conductor temperatures [5].
- Anemometer for measuring the wind speed [6].
- Light sensor for solar irradiance helps us estimate how much heat the sun adds to the conductor [8].
- An LM1602C LCD displays the sensor readings in real time for local monitoring [7].

All sensors are mounted on a bench-top test stand, ensuring safe and repeatable measurements. The Arduino receives sensor signals to perform signal filtering and scaling before it transmits the processed data through USB serial communication to a host computer for additional analysis.

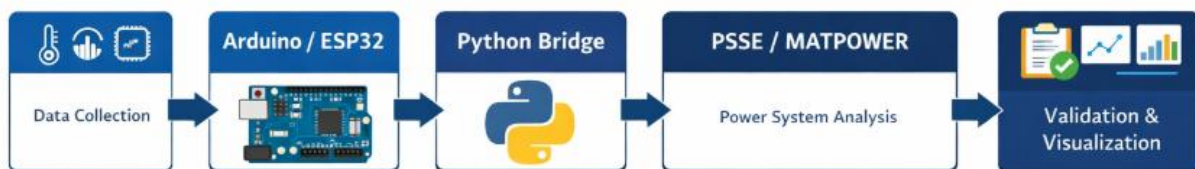


Figure 2: Block diagram of the hardware and data flow system.

The system implements RC low pass filters together with software averaging to produce precise and dependable measurement results and correcting grounding. All sensors received calibration through reference standards which resulted in temperature accuracy of $\pm 0.5^\circ\text{C}$ and wind speed accuracy of $\pm 0.5\text{ m/s}$ [5] [6].

Software and Computational Modeling

The software operates the IEEE 738 thermal balance model to deliver instant conductor ampacity calculations through its real time system [2].

- The Python/C++ script analyzes sensor information to determine DLR values for every power line while following IEEE 738 [2] standards to calculate Joule heating and solar gain and convective cooling and radiative losses.
- Temperature dependent conductor resistance is modeled as:

$$R(T_c) = R_{ref} \times [1 + \alpha(T_c - T_{ref})]$$

The script updates DLR values every second, enabling near real time monitoring and integration with power flow simulations.

Real Time Integration with PSSE/MATPOWER

The Python bridge sends DLR value updates to PSSE and MATPOWER through its continuous real time updating process [3]. The system automatically modifies transmission line thermal limits through these updates which let users match SLR and DLR line loading values while performing N-1 contingency analysis. The system records all processed data that appears on dashboards for monitoring and validation purposes.

Circuit Diagram

The prototype wiring ensures stable signal acquisition:

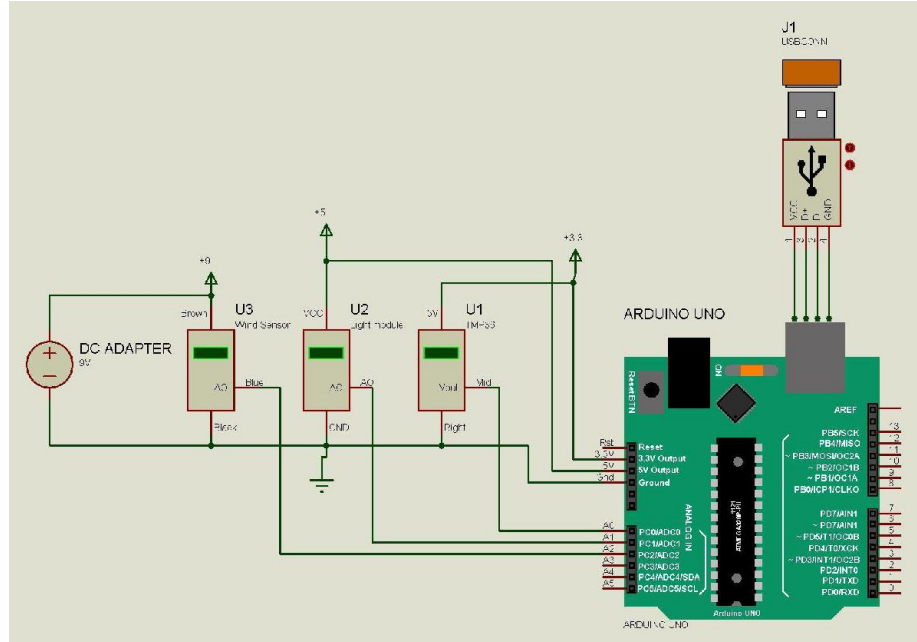


Figure 3: Circuit diagram of the hardware prototype showing sensor connections to Arduino UNO.

Validation and Testing

The system was verified through functional, performance, and integration tests:

- **Sensor Accuracy:** All sensors deliver measurements which stay within the boundaries defined by the manufacturer [5][6][8].
- **DLR Calculation:** The validation process for ampacity calculations using benchmark cases revealed percentage differences which remained under 10% [2].
- **PSSE Integration:** The system verified that real-time DLR data updates successfully modified line thermal restrictions and produced suitable system responses [3].
- **Hardware in the Loop Testing:** Lab tests varied environmental conditions to verify the system's dynamic response [5][6][8].
- **Overall System Reliability:** The system achieved $\geq 95\%$ uptime with less than 1% data loss during a continuous operation [3].

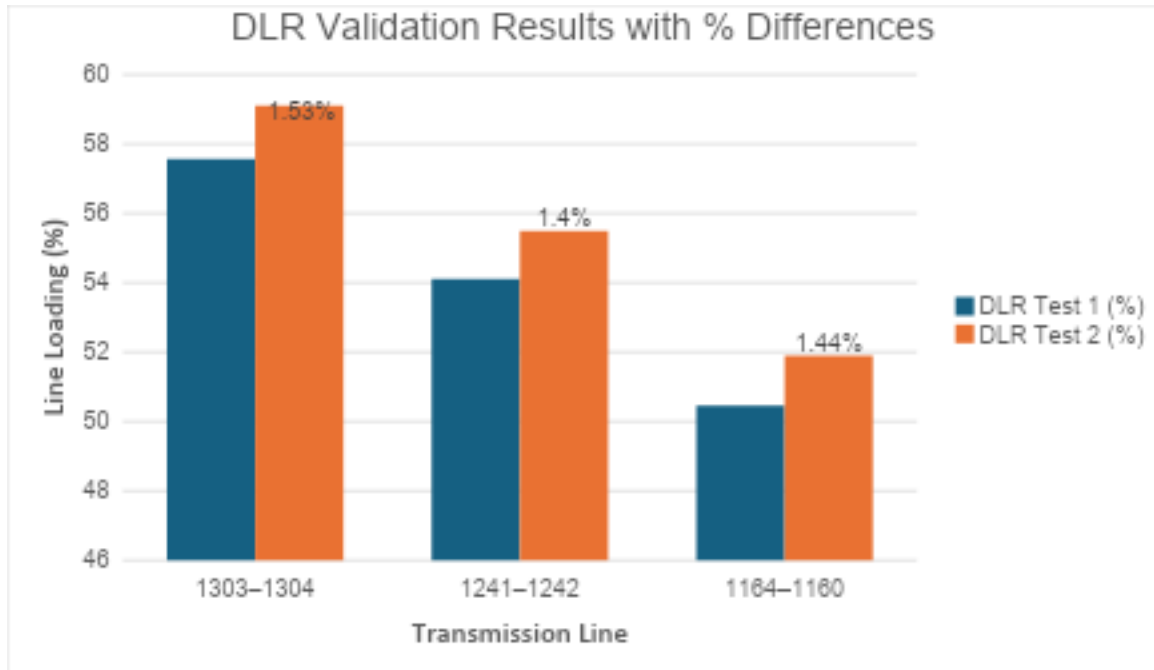


Figure 4: DLR validation results for three 138 kV lines.

The annotated percentages above the bars indicate the differences between DLR Test 1 and Test 2, highlighting system performance during validation.

Product Scope and Final Product Functionality

The Dynamic Line Rating (DLR) system developed in this project provides real time monitoring and assessment of transmission line capacity for selected 138 kV lines in Alberta. The system tracks environmental conditions while it performs conductor ampacity calculations which are used to assess power system performance through simulation data evaluation.

Scope of the Product

- The system monitors temperature, wind speed and solar irradiance through sensors which connect to an Arduino or ESP32 microcontroller.
- The IEEE 738 thermal model operates through Python/C++ code, to process live data which produces dynamic line ratings (DLR) for every power line.
- These DLR values are then integrated with PSSE and MATPOWER to simulate power flows and support N-1 contingency analysis.

- The research focuses on controlled laboratory experiments which study three specific 138 kV power lines, but it does not extend to complete system implementation or automatic system interference or database information retention.

Functional Description

The system works as follows:

- Environmental Sensing: The system collects immediate environmental data through sensors, which measure temperature and wind and light conditions.
- Data Acquisition: The Arduino or ESP32 system obtains sensor data which it processes through filtering before sending the results to a host computer system.
- DLR Calculation: Python/C++ scripts run IEEE 738 equations to generate real time conductor ampacity values from the available data.
- The DLR system updates thermal limits automatically which PSSE then MATPOWER use to perform operational assessments and contingency analysis.
- Monitoring and Validation: The system shows output data through dashboards which enable users to track information in real-time while verifying

Key Features

- The system performs real time DLR calculations every second to deliver immediate operational data.
- Operators can track line loadings through visual displays, which show them in relation to standard SLR values.
- The system performs automatic integration with PSSE and MATPOWER to support its operational analysis functions.

System Limitations

- The testing of sensors and prototype hardware occurs at laboratory scale but their deployment in the field needs equipment which can withstand harsh conditions.
- Only a subset of transmission lines three 138 kV lines is monitored in this project.

- Accuracy depends on the sensor precision and environmental conditions; measurement errors can affect the DLR calculations.

Impact of Functionality

The DLR system offers several tangible benefits, including:”

- Operators can reduce power line congestion through their ability to make immediate adjustments to thermal operating boundaries.
- The system enables better incorporation of renewable power sources by supporting increased power flow through lines when weather conditions are optimal.
- The system includes a foundation which supports upcoming network growth through its capability to extend the method for additional transmission lines and expanded network sections

Technical Specifications of the Final Product

The Dynamic Line Rating (DLR) system functions as a laboratory scale prototype which allows operators to track conductor ampacity in real time for specific 138 kV transmission lines. The technical specifications document specifies all system components which include hardware elements and software programs and operational standards that define the system's operational abilities.

Table 1: Comparison of SLR and DLR line loading for selected 138 kV transmission lines

Line ID	SLR Loading (%)	DLR Loading (%)	% Reduction
1303–1304	74.84	57.57	17.27
1241–1242	70.33	54.10	16.23
1164–1160	65.60	50.46	15.14

Hardware Specifications

- Microcontroller: The Arduino UNO and ESP32 run at 16 MHz and provide 32 KB of flash memory while they process sensors and send data immediately [4].
- Temperature Sensor: The TMP36 sensor operates between -40°C and $+125^{\circ}\text{C}$ while maintaining an average measurement precision of $\pm 0.5^{\circ}\text{C}$ [5].

- Anemometer: The Adafruit 1733 anemometer tracks wind velocity between 0 and 50 m/s while maintaining an accuracy level of ± 0.5 m/s [6].
- Light Sensor: A sensor based on LDR and photodiode technology measures light levels between 0 and 1000 lux to determine solar heat gain [8].
- LCD Display: The LM1602C display offers 16×2-character space [7].
- Power Supply: The system requires a 5-volt regulated power source which provides power to both the Arduino and sensors, while sharing a common ground and utilizing decoupling capacitors to minimize electrical interference.

Software Specifications

- DLR Calculation: Python and C++ codes follow IEEE Std 738-2012 to calculate real time conductor ampacity through environmental data processing [2].
- Sampling Rate: 1 Hz (one update per second) for real time computation and seamless integration with PSSE and MATPOWER.
- Integration Interface: A Python bridge communicates DLR values to PSSE and MATPOWER, automatically updating line thermal limits [3][8].
- Validation Scripts: The automated scripts verify DLR results by comparing them to benchmark cases and laboratory test data [2][3].

Operational Specifications

- Monitored Lines: Three selected 138 kV transmission lines in the Alberta transmission network test system.
- Measurement Accuracy:
 - Temperature ± 0.5 °C
 - Wind ± 0.5 m/s
 - DLR calculation deviation $\leq 10\%$ compared to IEEE benchmark values [2].
- System Reliability: Laboratory testing indicates $\geq 95\%$ uptime with $< 1\%$ data loss during continuous operation [3].

Deviations from Design Roadmap

- The system is implemented as a small-scale prototype, while a full deployment would cover a larger network with field-grade sensors.
- PSSE and MATPOWER integration occurs through laboratory work which operates separately from the active power system network.
- The LCD display shows current local data, which was not part of the initial plan, but it improves our ability to monitor and verify the system.

Measuring Success and Validation Test Results

The Dynamic Line Rating (DLR) system underwent evaluation to verify its compliance with project goals which demanded precise environmental data collection and proper dynamic conductor capacity calculation and dependable system linkage between PSSE and MATPOWER for operational analysis. The validation process consisted of five main parts that included sensor accuracy and DLR computation and software integration and hardware in the loop testing and complete system performance evaluation.

Sensor Accuracy

Each sensor was tested to verify that measurements fell within manufacturer-specified ranges:

- Temperature (TMP36): ± 0.5 °C [5]
- Wind (Adafruit 1733 Anemometer): 0–50 m/s with ± 0.5 m/s accuracy [6]
- Light sensor: 0–1000 lux [8]

The sensors underwent calibration through reference measurements which produced stable sensor readings that fell within the established acceptable range during multiple testing sessions.

DLR Computation

The Python/C++ implementation of IEEE 738 was validated against benchmark calculations and showing deviations of $\leq 10\%$ [2]. The system provides precise real time calculations for conductor ampacity through its environmental data processing capabilities.

PSSE Integration

DLR values were successfully integrated with PSSE and MATPOWER in the laboratory test system.

- The thermal limits of transmission lines received proper adjustments through real time updates which also passed N-1 contingency tests to prove system stability during the operational N-1 events [3].
- All updates were logged to ensure reproducibility and facilitate review.

The following PSSE validation output confirms that all bus voltages remain within 1% of benchmark values:

Table 2: Comparison of SLR and DLR line loading for selected 138 kV transmission lines

BUS	Benchmark PU	Test PU	Error %	Status
1241	1.070103	1.070103	0.0000	PASS
1242	1.072509	1.072509	0.0000	PASS

The table shows that DLR reduces line loading by 15–17%, providing additional thermal headroom and supporting more efficient transmission operations.

Hardware in the Loop Testing

Laboratory tests varied environmental conditions to evaluate the system's response.

- Temperature, wind, and solar irradiance were systematically adjusted.
- The DLR outputs responded dynamically to each change, demonstrating proper hardware and software integration.

Overall System Reliability

- The system operated nonstop during multiple testing sessions which produced an uptime rate of at least 95% [3].
- The system maintained under 1% data loss during serial communication and logging operations which resulted in dependable real time system functionality.

Validation Figures

Figure 5: The DLR validation chart contains annotations which display how different test scenarios reduce power line stress for every 138 kV transmission line. The percentages above the DLR bars indicate how the system increases thermal headroom compared to SLR benchmarks.

Figure 5: The DLR validation chart contains annotations which display how different test scenarios reduce power line stress for every 138 kV transmission line. The percentages above the DLR bars indicate how the system increases thermal headroom compared to SLR benchmarks.

This validation confirms that the DLR system:

- The system provides exact measurements for all environmental conditions.
- The system calculates conductor ampacity values which remain within the defined range of acceptable errors.
- The system achieves dependable integration capabilities for power system simulation applications.
- The system performs active responses to all variations which occur in its environmental setting.

The project goals have been met by the system which also delivers a repeatable framework for academic research and operational use.

List of Tools, Materials, Supplies, and Costs

The following list summarizes all tools, hardware, software, and supplies that were used to develop and validate the Dynamic Line Rating (DLR) system, along with their estimated costs. The presentation will demonstrate the current design roadmap against the first version to show all changes which took place.

Hardware Components

Table 3: Hardware Components

Item	Quantity	Purpose	Approx. Cost (CAD)
Arduino UNO/ESP32	1	Microcontroller for data acquisition and processing	40
TMP36 Temperature Sensor	1	Measures conductor/ambient temperature	5
Anemometer (Adafruit 1733)	1	Measures wind speed	50
Light Sensor (LDR/photodiode)	1	Measures solar irradiance	5
LCD Display (LM1602C)	1	Real-time local visualization	10
Breadboard & jumper wires	1 set	Hardware prototyping and connectivity	15
Resistors, capacitors, power supply	Assorted	Signal conditioning, power regulation	10

Total Hardware Cost: 135 CAD

Software and Tools

Table 4: Software and Tools

Item	Purpose	Approx. Cost
Lab power supply	Provides regulated 5 V for Arduino and sensors	50 CAD
Multimeter	Voltage/current measurements for validation	80 CAD
Mounting hardware	Stands, clips, sensor supports	20 CAD

Comparison with Design Roadmap

- The initial design plan included matching hardware components, but developers introduced LCD displays to improve the live monitoring which raised the project budget by a small amount.
- The project required breadboards and jumper wires to enable quick prototyping although these items were missing from the initial project documentation.
- The software expenses stayed at their planned levels because Python and MATPOWER exist as free software or can be accessed through academic licensing agreements.

Summary

- The total cost for the lab scale prototype was approximately 485 CAD.
- All materials and tools provided full functionality of the DLR system for validation and performance testing.
- The system works as a scalable solution which uses its current components to build bigger experimental systems and operational field systems.

Reflections

Ethical Considerations

Scientists must maintain ethical standards during Dynamic Line Rating system development and testing, because they evaluate the system through their lab scale prototype work.

The project establishes safety protocols which protect sensor data collection and simulation activities from creating any dangerous situations. The testing process took place within secure facilities which safeguarded both staff members and their equipment. The system architecture includes grid protection systems which defend electrical power lines from excessive current that could endanger residential communities. The system operates to enhance electrical grid dependability through its protection against line overloading which safeguards residential communities that depend on this infrastructure.

Societal and Environmental Impact

The DLR system delivers clear societal and environmental benefits. The system allows instant monitoring which optimizes power distribution to stop renewable energy resources from being needlessly curtailed. The system generates environmentally friendly power which results in decreased greenhouse gas emissions while Alberta advances its sustainable electricity generation goals. The improved reliability of transmission lines creates two positive effects for communities, because it stops power outages while delivering steady power access to all customers including homes, businesses and essential facilities.

Project Management

The project needed precise organization of planning activities with effective teamwork and schedule control to achieve every milestone that included the Project Proposal and Design Roadmap and PR2. The work process divided into four distinct phases which included hardware development and software programming and simulation and validation activities to help team members concentrate on their designated goals. The team used an iterative testing approach with hardware in the loop validation to discover and correct errors which proved their project management abilities.

Economics

The system was developed with cost efficiency in mind. The developers created their prototype system by combining standard sensors with Arduino and ESP32 microcontrollers which cost them around 485 CAD to build while saving them money compared to buying industrial equipment. The system proves that advanced monitoring solutions exist for affordable prices which allow researchers and pilot programs and upcoming utility operations to access DLR technology.

Lifelong Learning

The DLR system development process enabled me to acquire knowledge which would benefit my entire professional career. Team members deepened their understanding of power system modeling, real time data acquisition, and software integration with PSSE and MATPOWER. The project showed how essential it is to merge IEEE 738 standards knowledge with hands-on laboratory work because it enables the team to develop essential engineering abilities for their professional development. The

research findings about sensor calibration and debugging and real-time integration will direct all upcoming research activities and professional work in this field.

Appendix

Appendix A- Figures

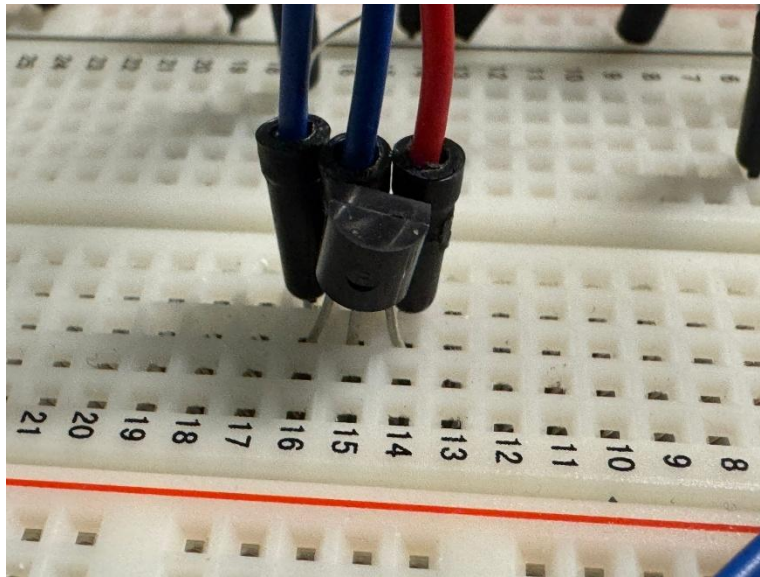


Figure 5-Temperature Sensor

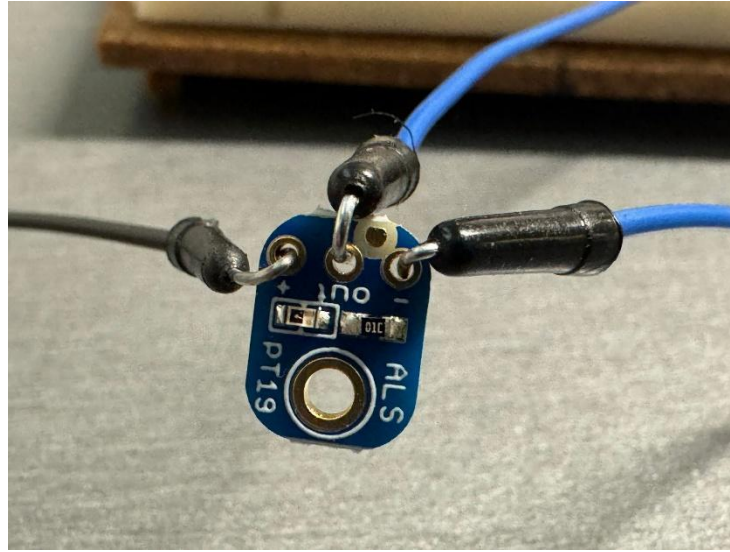


Figure 6-Light Sensor

Appendix - Python Code

Parse_Report:

```
report_file = report_file = r"C:\Users\kibria.muttakin\Documents\DLR Project\report-allbus.txt"

current_bus = None

current_bus_name = None

current_bus_kv = None

results = []

with open(report_file, "r", encoding="utf-8", errors="ignore") as f:

    for line in f:

        s = line.strip()

        # Detect BUS header

        # Example:

        # BUS 76502 B23S_01_2 34.500 CKT ...

        if s.startswith("BUS"):

            parts = s.split()
```

```

if len(parts) >= 5 and parts[0] == "BUS":
    try:
        current_bus = int(parts[1])
        current_bus_kv = float(parts[3])
        current_bus_name = parts[2]
    except:
        current_bus = None
        current_bus_name = None
        current_bus_kv = None
    continue

# Detect branch rows

# Example:
# TO 533 VICTORIA 69.000 21 12.6 1.8 12.7 17 ...

if s.startswith("TO") and current_bus is not None:
    parts = s.split()

# Skip non-branch rows

if len(parts) < 9:
    continue

if parts[1] in ["LOAD-PQ", "GENERATION", "SWITCHED"]:
    continue

if not parts[1].isdigit():
    continue

try:
    to_bus = int(parts[1])
    to_name = parts[2]
    to_kv = float(parts[3])
    ckt = parts[4]

```

```

mw = float(parts[5])
mvar = float(parts[6])
mva = float(parts[7])
pct = float(parts[8])

# Keep transmission-level branches only
if current_bus_kv >= 69 and to_kv >= 69:
    results.append({
        "from_bus": current_bus,
        "from_name": current_bus_name,
        "from_kv": current_bus_kv,
        "to_bus": to_bus,
        "to_name": to_name,
        "to_kv": to_kv,
        "ckt": ckt,
        "mw": mw,
        "mvar": mvar,
        "mva": mva,
        "pct": pct
    })
except:
    continue

# Sort by highest loading
results.sort(key=lambda x: x["pct"], reverse=True)
print(f"Total candidate branches found: {len(results)}\n")
print("Top 30 loaded branches:\n")
for r in results[:30]:
    print(

```



```
f"{r['from_bus']}{r['from_name']}{r['from_kv']} kV) -> "
f"{r['to_bus']}{r['to_name']}{r['to_kv']} kV), "
f"ckt={r['ckt']}, MVA={r['mva']}, loading={r['pct']}%"
)
```

Check_CSV

```
import pandas as pd

excel_file = r"C:\Users\kibria.muttakin\Documents\DLR Project\dlr_log_20260224_172353.xlsx"

df = pd.read_excel(excel_file)

print("COLUMNS:")

print(df.columns.tolist())

print()

print("MAX dynamic_ampacity_A =", df["dynamic_ampacity_A"].max())

print()

top10 = df.sort_values("dynamic_ampacity_A", ascending=False)[
    ["timestamp", "temp_c", "light_percent", "wind_ms", "dynamic_ampacity_A", "static_ampacity_A",
    "extra_margin_A"]
].head(10)

print("TOP 10 ROWS BY dynamic_ampacity_A:")

print(top10.to_string(index=False))
```

Read_Branch_Data

```
import os

import sys

# ===== EDIT THESE =====

PSSE_PY_PATH = r"C:\Program Files\PTI\PSSE35\35.0\PSSPY37"

PSSE_BIN_PATH = r"C:\Program Files\PTI\PSSE35\35.0\PSSBIN"

sav_file = r"C:\Users\kibria.muttakin\Documents\DLR Project\2021RC_2025SP_South-High_Tie-
Econ_Solar-0.95_p.sav"

sys.path.append(PSSE_PY_PATH)
```

```

os.environ["PATH"] += ";" + PSSE_BIN_PATH

import psspy

import redirect

redirect.psse2py()

psspy.psseinit(14000)

FROM_BUS = 1303

TO_BUS = 1304

CKT_ID = "05"

ierr = psspy.case(sav_file)

print("Open case ierr =", ierr)

# read branch integer data

for fld in ["STATUS", "METBUS", "NMETBUS", "OWN1", "OWN2", "OWN3", "OWN4"]:

    try:

        ierr, val = psspy.brnint(FROM_BUS, TO_BUS, CKT_ID, fld)

        print(f"{fld}: ierr={ierr}, val={val}")

    except Exception as e:

        print(f"{fld}: failed -> {e}")

# read branch real data

for fld in ["R", "X", "B", "RATEA", "RATEB", "RATEC", "LEN"]:

    try:

        ierr, val = psspy.brndat(FROM_BUS, TO_BUS, CKT_ID, fld)

        print(f"{fld}: ierr={ierr}, val={val}")

    except Exception as e:

        print(f"{fld}: failed -> {e}")

```

Apply_Dlr_Safe

```

import os

import sys

```

```

import math

import pandas as pd

# =====

# 1. EDIT THESE PSSE PATHS

# =====

PSSE_PY_PATH = r"C:\Program Files\PTI\PSSE35\35.0\PSSPY37"
PSSE_BIN_PATH = r"C:\Program Files\PTI\PSSE35\35.0\PSSBIN"

sys.path.append(PSSE_PY_PATH)

os.environ["PATH"] += ";" + PSSE_BIN_PATH

import psspy

import redirect

redirect.psse2py()

psspy.psseinit(14000)

# Default placeholders

_i = psspy.getdefaultint()
_f = psspy.getdefaultreal()
_s = psspy.getdefaultchar()

# =====

# 2. EDIT THESE FILE PATHS

# =====

sav_file = r"C:\Users\kibria.muttakin\Documents\DLR Project\2021RC_2025SP_South-High_Tie-
Econ_Solar-0.95_p.sav"

excel_file = r"C:\Users\kibria.muttakin\Documents\DLR Project\dlr_log_20260224_172353.xlsx"

out_file = r"C:\Users\kibria.muttakin\Documents\DLR Project\case_dlr_1303_1304_fixed.sav"

# =====

# 3. CHOSEN LINE

# =====

FROM_BUS = 1303

```

```

TO_BUS = 1304

CKT_ID = "05"

LINE_KV = 138.0

# Exact PSSE rates from your branch-data output

RATEA_OLD = 157.1666717529297

RATEB_OLD = 190.70834350585938

RATEC_OLD = 157.1666717529297

# Cap DLR at 130% of original RATEA

MAX_DLR_RATE_MVA = RATEA_OLD * 1.30

# =====

# 4. READ EXCEL AND PICK BEST ROW

# =====

df = pd.read_excel(excel_file)

best_row = df.sort_values("dynamic_ampacity_A", ascending=False).iloc[0]

dyn_ampacity_A = float(best_row["dynamic_ampacity_A"])

print("Best dynamic ampacity (A) =", dyn_ampacity_A)

# =====

# 5. CONVERT A TO MVA

# =====

dyn_mva = math.sqrt(3) * LINE_KV * (dyn_ampacity_A / 1000.0)

new_rate_mva = min(dyn_mva, MAX_DLR_RATE_MVA)

print("Old RATEA =", round(RATEA_OLD, 3))

print("Old RATEB =", round(RATEB_OLD, 3))

print("Old RATEC =", round(RATEC_OLD, 3))

print("Raw DLR MVA =", round(dyn_mva, 3))

print("New capped DLR rate =", round(new_rate_mva, 3))

# =====

```

```
# 6. OPEN CASE
```

```
# =====
```

```
ierr = psspy.case(sav_file)
```

```
print("Open case ierr =", ierr)
```

```
ierr = psspy.fnsf([0,0,0,1,1,0,99,0])
```

```
print("Base solve ierr =", ierr)
```

```
# =====
```

```
# 7. CHANGE ONLY THE RATINGS
```

```
# Leave all other values unchanged using defaults
```

```
# =====
```

```
intgar = [_i, _i, _i, _i, _i, _i]
```

```
realar = [_f, _f, _f, new_rate_mva, new_rate_mva, new_rate_mva, _f, _f, _f, _f, _f, _f]
```

```
ierr = psspy.branch_chng_3(FROM_BUS, TO_BUS, CKT_ID, intgar, realar)
```

```
print("branch_chng_3 ierr =", ierr)
```

```
# =====
```

```
# 8. SOLVE AGAIN
```

```
# =====
```

```
ierr = psspy.fnsf([0,0,0,1,1,0,99,0])
```

```
print("DLR solve ierr =", ierr)
```

```
# =====
```

```
# 9. SAVE NEW CASE
```

```
# =====
```

```
ierr = psspy.save(out_file)
```

```
print("Save ierr =", ierr)
```

```
print("DONE. New case saved.")
```

Compare_Cases

```
import os
```

```

import sys

PSSE_PY_PATH = r"C:\Program Files\PTI\PSSE35\35.0\PSSPY37"

PSSE_BIN_PATH = r"C:\Program Files\PTI\PSSE35\35.0\PSSBIN"

base_case = r"C:\Users\kibria.muttakin\Documents\DLR Project\2021RC_2025SP_South-High_Tie-
Econ_Solar-0.95_p.sav"

dlr_case = r"C:\Users\kibria.muttakin\Documents\DLR Project\case_dlr_1303_1304_final.sav"

FROM_BUS = 1303

TO_BUS = 1304

CKT_ID = "05"

sys.path.append(PSSE_PY_PATH)

os.environ["PATH"] += ";" + PSSE_BIN_PATH

import psspy

import redirect

redirect.psse2py()

psspy.psseinit(14000)

def get_data(casefile, label):

    ierr = psspy.case(casefile)

    print(f"\n{label} open ierr =", ierr)

    ierr = psspy.fnsf([0,0,0,1,1,0,99,0])

    print(f"{label} solve ierr =", ierr)

    ierr, ratea = psspy.brndat(FROM_BUS, TO_BUS, CKT_ID, "RATEA")

    ierr, rateb = psspy.brndat(FROM_BUS, TO_BUS, CKT_ID, "RATEB")

    ierr, ratec = psspy.brndat(FROM_BUS, TO_BUS, CKT_ID, "RATEC")

    ierr, p = psspy.brnmisc(FROM_BUS, TO_BUS, CKT_ID, "P")

    ierr, q = psspy.brnmisc(FROM_BUS, TO_BUS, CKT_ID, "Q")

    ierr, pct = psspy.brnmisc(FROM_BUS, TO_BUS, CKT_ID, "PCTMVA")

    mva = (p**2 + q**2) ** 0.5

    print(f"{label} RATEA = {ratea}")

```

```

    print(f"{label} RATEB = {rateb}")
    print(f"{label} RATEC = {ratec}")
    print(f"{label} P = {p}")
    print(f"{label} Q = {q}")
    print(f"{label} MVA = {mva}")
    print(f"{label} Loading % = {pct}")

    return ratea, mva, pct

base_rate, base_mva, base_pct = get_data(base_case, "BASE")
dlr_rate, dlr_mva, dlr_pct = get_data(dlr_case, "DLR")

print("\n===== FINAL COMPARISON =====")

print("Base RATEA  =", base_rate)
print("DLR RATEA  =", dlr_rate)
print("Base MVA    =", base_mva)
print("DLR MVA     =", dlr_mva)
print("Base Loading =", base_pct)
print("DLR Loading =", dlr_pct)

```

All_Lines_DLR

```

import os

import sys

import math

import pandas as pd

# =====

# 1) EDIT THESE PATHS

# =====

PSSE_PY_PATH = r"C:\Program Files\PTI\PSSE35\35.0\PSSPY37"

PSSE_BIN_PATH = r"C:\Program Files\PTI\PSSE35\35.0\PSSBIN"

BASE_CASE = r"C:\Users\kibria.muttakin\Documents\DLR Project\2021RC_2025SP_South-High_Tie-
Econ_Solar-0.95_p.sav"

```

```

EXCEL_FILE = r"C:\Users\kibria.muttakin\Documents\DLR Project\dlr_log_20260224_172353.xlsx"

OUTPUT_FOLDER = r"C:\Users\kibria.muttakin\Documents\DLR Project"

# =====

# 2) LINES TO STUDY

# =====

LINES = [

    {"from_bus": 1303, "to_bus": 1304, "ckt": "05", "kv": 138.0, "name": "1303-1304-05"},

    {"from_bus": 1241, "to_bus": 1242, "ckt": "15", "kv": 138.0, "name": "1241-1242-15"},

    {"from_bus": 1164, "to_bus": 1160, "ckt": "32", "kv": 138.0, "name": "1164-1160-32"},

]

# =====

# 3) PSSE SETUP

# =====

sys.path.append(PSSPY_PATH)

os.environ["PATH"] += ";" + PSSE_BIN_PATH

import psspy

import redirect

redirect.psse2py()

psspy.psseinit(14000)

_i = psspy.getdefaultint()

# =====

# 4) READ BEST DLR VALUE FROM EXCEL

# =====

df = pd.read_excel(EXCEL_FILE)

best_row = df.sort_values("dynamic_ampacity_A", ascending=False).iloc[0]

dyn_ampacity_A = float(best_row["dynamic_ampacity_A"])

print("Best dynamic ampacity from Excel (A) =", dyn_ampacity_A)

```



```

print()

# =====

# 5) HELPER FUNCTIONS

# =====

def open_and_solve(casefile, label):

    ierr = psspy.case(casefile)

    print(f"{label} open ierr =", ierr)

    ierr = psspy.fnsf([0, 0, 0, 1, 1, 0, 99, 0])

    print(f"{label} solve ierr =", ierr)

    return ierr

def get_branch_results(from_bus, to_bus, ckt):

    ierr, ratea = psspy.brndat(from_bus, to_bus, ckt, "RATEA")

    ierr, rateb = psspy.brndat(from_bus, to_bus, ckt, "RATEB")

    ierr, ratec = psspy.brndat(from_bus, to_bus, ckt, "RATEC")

    ierr, p = psspy.brnmisc(from_bus, to_bus, ckt, "P")

    ierr, q = psspy.brnmisc(from_bus, to_bus, ckt, "Q")

    ierr, pct = psspy.brnmisc(from_bus, to_bus, ckt, "PCTMVA")

    mva = (p**2 + q**2) ** 0.5

    return {

        "ratea": ratea,

        "rateb": rateb,

        "ratec": ratec,

        "p": p,

        "q": q,

        "mva": mva,

        "pct": pct

    }

```

```

def apply_dlr_to_line(base_case, line, dyn_ampacity_A, output_folder):

    from_bus = line["from_bus"]

    to_bus = line["to_bus"]

    ckt = line["ckt"]

    kv = line["kv"]

    name = line["name"]

    print("=" * 70)

    print(f"Processing line: {name}")

    # ---- Base case results ----

    open_and_solve(base_case, "BASE")

    base_res = get_branch_results(from_bus, to_bus, ckt)

    print("Base RATEA =", base_res["ratea"])

    print("Base RATEB =", base_res["rateb"])

    print("Base RATEC =", base_res["ratec"])

    print("Base MVA  =", base_res["mva"])

    print("Base %Load =", base_res["pct"])

    # ---- Read branch electrical data ----

    ierr, rx = psspy.brndt2(from_bus, to_bus, ckt, "RX")

    if ierr != 0:

        raise RuntimeError(f"Could not read RX for line {name}, ierr={ierr}")

    r = rx.real

    x = rx.imag

    ierr, charg = psspy.brndat(from_bus, to_bus, ckt, "CHARG")

    if ierr != 0:

        raise RuntimeError(f"Could not read CHARG for line {name}, ierr={ierr}")

    # ---- Convert DLR A -> MVA ----

    dyn_mva = math.sqrt(3) * kv * (dyn_ampacity_A / 1000.0)

```

```

# Cap at 130% of original RATEA
new_rate = min(dyn_mva, base_res["ratea"] * 1.30)

print("Raw DLR MVA    =", dyn_mva)
print("Capped DLR RATEA =", new_rate)

# ---- Re-open base case before modification ----
open_and_solve(base_case, "RELOAD")

intgar = [_i, _i, _i, _i, _i, _i]
realar = [r, x, charg, new_rate, new_rate, new_rate]

ierr = psspy.branch_chng(from_bus, to_bus, ckt, intgar, realar)
print("branch_chng ierr =", ierr)

if ierr != 0:
    raise RuntimeError(f"branch_chng failed for line {name}, ierr={ierr}")

# ---- Confirm new ratings ----
ierr, new_ratea = psspy.brndat(from_bus, to_bus, ckt, "RATEA")
ierr, new_rateb = psspy.brndat(from_bus, to_bus, ckt, "RATEB")
ierr, new_ratec = psspy.brndat(from_bus, to_bus, ckt, "RATEC")

print("NEW RATEA =", new_ratea)
print("NEW RATEB =", new_rateb)
print("NEW RATEC =", new_ratec)

# ---- Solve DLR case ----
ierr = psspy.fnsf([0, 0, 0, 1, 1, 0, 99, 0])
print("DLR solve ierr =", ierr)

if ierr != 0:
    raise RuntimeError(f"DLR solve failed for line {name}, ierr={ierr}")

dlr_res = get_branch_results(from_bus, to_bus, ckt)
print("DLR RATEA =", dlr_res["ratea"])

```

```

print("DLR MVA =", dlr_res["mva"])
print("DLR %Load =", dlr_res["pct"])
# ---- Save line-specific output case ----
out_file = os.path.join(output_folder, f"case_dlr_{name}.sav")
ierr = psspy.save(out_file)
print("Save ierr =", ierr)
if ierr != 0:
    raise RuntimeError(f"Save failed for line {name}, ierr={ierr}")
return {
    "line": name,
    "from_bus": from_bus,
    "to_bus": to_bus,
    "ckt": ckt,
    "kv": kv,
    "base_ratea": base_res["ratea"],
    "base_mva": base_res["mva"],
    "base_pct": base_res["pct"],
    "dlr_ratea": dlr_res["ratea"],
    "dlr_mva": dlr_res["mva"],
    "dlr_pct": dlr_res["pct"],
    "saved_case": out_file
}

# =====
# 6) RUN FOR ALL LINES
# =====

summary = []

```

```

for line in LINES:

    try:

        result = apply_dlr_to_line(BASE_CASE, line, dyn_ampacity_A, OUTPUT_FOLDER)

        summary.append(result)

    except Exception as e:

        print(f"\nERROR for line {line['name']}: {e}\n")

# =====

# 7) FINAL SUMMARY TABLE

# =====

print("\n" + "=" * 70)

print("FINAL SUMMARY")

print("=" * 70)

if summary:

    print(f"{'Line':<18} {'kV':>6} {'Base RATEA':>12} {'DLR RATEA':>12} {'Base %':>10} {'DLR %':>10}")

    for r in summary:

        print(

            f"{r['line']:<18} "

            f"{r['kv']:>6.1f} "

            f"{r['base_ratea']:>12.2f} "

            f"{r['dlr_ratea']:>12.2f} "

            f"{r['base_pct']:>10.2f} "

            f"{r['dlr_pct']:>10.2f}"

        )

    print("\nSaved cases:")

    for r in summary:

        print(r["saved_case"])

else:

```

```
print("No successful line results.")
```

Contingency_Test

```
import os
```

```
import sys
```

```
# =====
```

```
# 1) EDIT THESE PATHS
```

```
# =====
```

```
PSSE_PY_PATH = r"C:\Program Files\PTI\PSSE35\35.0\PSSPY37"
```

```
PSSE_BIN_PATH = r"C:\Program Files\PTI\PSSE35\35.0\PSSBIN"
```

```
BASE_CASE = r"C:\Users\kibria.muttakin\Documents\DLR Project\2021RC_2025SP_South-High_Tie-  
Econ_Solar-0.95_p.sav"
```

```
DLR_CASE = r"C:\Users\kibria.muttakin\Documents\DLR Project\case_dlr_1303-1304-05.sav"
```

```
BASE_OUTAGE_CASE = r"C:\Users\kibria.muttakin\Documents\DLR  
Project\base_case_outage_1241_1242_15.sav"
```

```
DLR_OUTAGE_CASE = r"C:\Users\kibria.muttakin\Documents\DLR  
Project\dlr_case_outage_1241_1242_15.sav"
```

```
# =====
```

```
# 2) TARGET LINE = line you want to monitor
```

```
# =====
```

```
TARGET_FROM = 1303
```

```
TARGET_TO = 1304
```

```
TARGET_CKT = "05"
```

```
# =====
```

```
# 3) OUTAGE LINE = milder candidate
```

```
# =====
```

```
OUT_FROM = 1241
```

```
OUT_TO = 1242
```

```

OUT_CKT = "15"

# =====

# 4) PSSE SETUP

# =====

sys.path.append(P SSE_PY_PATH)
os.environ["PATH"] += ";" + PSSE_BIN_PATH

import psspy

import redirect

redirect.psse2py()

psspy.psseinit(14000)

_i = psspy.getdefaultint()
_f = psspy.getdefaultreal()

# =====

# 5) HELPER FUNCTIONS

# =====

def open_and_solve(casefile, label):

    ierr = psspy.case(casefile)

    print(f"\n{label} open ierr =", ierr)

    ierr = psspy.fnsf([0, 0, 0, 1, 1, 0, 99, 0])

    print(f"{label} solve ierr =", ierr)

    return ierr

def get_branch_results(from_bus, to_bus, ckt, label):

    ierr, ratea = psspy.brndat(from_bus, to_bus, ckt, "RATEA")

    ierr, rateb = psspy.brndat(from_bus, to_bus, ckt, "RATEB")

    ierr, ratec = psspy.brndat(from_bus, to_bus, ckt, "RATEC")

    ierr, p = psspy.brnmsc(from_bus, to_bus, ckt, "P")

    ierr, q = psspy.brnmsc(from_bus, to_bus, ckt, "Q")

```

```

ierr, pct = psspy.brnmisc(from_bus, to_bus, ckt, "PCTMVA")

mva = (p**2 + q**2) ** 0.5

print(f"{label} RATEA = {ratea}")
print(f"{label} RATEB = {rateb}")
print(f"{label} RATEC = {ratec}")

print(f"{label} P   = {p}")
print(f"{label} Q   = {q}")

print(f"{label} MVA  = {mva}")
print(f"{label} Load% = {pct}")

return {
    "ratea": ratea,
    "rateb": rateb,
    "ratec": ratec,
    "p": p,
    "q": q,
    "mva": mva,
    "pct": pct
}

def outage_branch_safely(from_bus, to_bus, ckt):
    ierr, rx = psspy.brndt2(from_bus, to_bus, ckt, "RX")

    if ierr != 0:
        raise RuntimeError(f"Could not read RX for outage line {from_bus}-{to_bus}-{ckt}, ierr={ierr}")

    r = rx.real
    x = rx.imag

    ierr, charg = psspy.brndat(from_bus, to_bus, ckt, "CHARG")

    if ierr != 0:
        raise RuntimeError(f"Could not read CHARG for outage line {from_bus}-{to_bus}-{ckt}, ierr={ierr}")

```



```

# First integer = STATUS, set to 0 for out of service

intgar = [0, _i, _i, _i, _i, _i]

realar = [r, x, charg, _f, _f, _f]

ierr = psspy.branch_chng(from_bus, to_bus, ckt, intgar, realar)

print(f"Outage branch_chng ierr = {ierr}")

return ierr

def run_contingency(casefile, savefile, label):

    open_and_solve(casefile, label)

    pre = get_branch_results(TARGET_FROM, TARGET_TO, TARGET_CKT, f"{label} PRE-OUTAGE TARGET")

    ierr = outage_branch_safely(OUT_FROM, OUT_TO, OUT_CKT)

    if ierr != 0:

        raise RuntimeError(f"Outage failed for {label}, ierr={ierr}")

    ierr = psspy.fnsi([0, 0, 0, 1, 1, 0, 99, 0])

    print(f"{label} POST-OUTAGE solve ierr =", ierr)

    post = get_branch_results(TARGET_FROM, TARGET_TO, TARGET_CKT, f"{label} POST-OUTAGE TARGET")

    ierr = psspy.save(savefile)

    print(f"{label} save ierr =", ierr)

    return pre, post

# =====

# 6) RUN BASE CASE CONTINGENCY

# =====

base_pre, base_post = run_contingency(BASE_CASE, BASE_OUTAGE_CASE, "BASE CASE")

# =====

# 7) RUN DLR CASE CONTINGENCY

# =====

dlr_pre, dlr_post = run_contingency(DLR_CASE, DLR_OUTAGE_CASE, "DLR CASE")

```

```

# =====
# 8) FINAL SUMMARY
# =====

print("\n" + "=" * 75)

print("FINAL CONTINGENCY SUMMARY")

print("=" * 75)

print("\nTarget line monitored: 1303 -> 1304, ckt 05")

print("Outaged line:      1241 -> 1242, ckt 15")

print("\n----- BASE CASE -----")

print(f"Pre-outage RATEA = {base_pre['ratea']}")
print(f"Pre-outage MVA   = {base_pre['mva']}")
print(f"Pre-outage Loading = {base_pre['pct']}")

print(f"Post-outage RATEA = {base_post['ratea']}")
print(f"Post-outage MVA   = {base_post['mva']}")
print(f"Post-outage Loading = {base_post['pct']}")

print("\n----- DLR CASE -----")

print(f"Pre-outage RATEA = {dlr_pre['ratea']}")
print(f"Pre-outage MVA   = {dlr_pre['mva']}")
print(f"Pre-outage Loading = {dlr_pre['pct']}")

print(f"Post-outage RATEA = {dlr_post['ratea']}")
print(f"Post-outage MVA   = {dlr_post['mva']}")
print(f"Post-outage Loading = {dlr_post['pct']}")

print("\n----- COMPARISON -----")

print("Base post-outage loading =", base_post['pct'])
print("DLR post-outage loading =", dlr_post['pct'])

```

Validation_DLR

```
import os
```

```

import sys

import math

# =====

# 1) EDIT THESE PATHS

# =====

PSSE_PY_PATH = r"C:\Program Files\PTI\PSSE35\35.0\PSSPY37"

PSSE_BIN_PATH = r"C:\Program Files\PTI\PSSE35\35.0\PSSBIN"

BENCHMARK_CASE = r"C:\Users\monjurul.haque\Documents\DLR Project\2021RC_2025SP_South-
High_Tie-Econ_Solar-0.95_p.sav"

TEST_CASE = r"C:\Users\monjurul.haque\Documents\DLR Project\case_dlr_1241-1242-15.sav"

# =====

# 2) CHOOSE WHAT TO VALIDATE

# =====

# Buses: voltage magnitude comparison

BUSES_TO_CHECK = [1241, 1242, 1164, 1160]

# Branches: MVA flow / loading / rating comparison

BRANCHES_TO_CHECK = [

    {"from_bus": 1241, "to_bus": 1242, "ckt": "15", "name": "1241-1242-15"},

    {"from_bus": 1164, "to_bus": 1160, "ckt": "32", "name": "1164-1160-32"},

]

THRESHOLD_PERCENT = 1.0

# =====

# 3) PSSE SETUP

# =====

sys.path.append(PSSE_PY_PATH)

os.environ["PATH"] += ";" + PSSE_BIN_PATH

import psspy

```

```

import redirect
redirect.psse2py()
psspy.psseinit(14000)

# =====

# 4) HELPERS

# =====

def pct_error(ref, test):
    """Relative percent error. Handles near-zero safely."""
    if abs(ref) < 1e-9:
        return 0.0 if abs(test) < 1e-9 else float("inf")
    return abs((test - ref) / ref) * 100.0

def open_and_solve(casefile, label):
    ierr = psspy.case(casefile)
    print(f"{label} open ierr = {ierr}")
    ierr = psspy.fns1([0, 0, 0, 1, 1, 0, 99, 0])
    print(f"{label} solve ierr = {ierr}")
    return ierr

def get_bus_voltage(busnum):
    ierr, pu = psspy.busdat(busnum, "PU")
    if ierr != 0:
        raise RuntimeError(f"Could not read bus voltage for bus {busnum}, ierr={ierr}")
    return pu

def get_branch_data(from_bus, to_bus, ckt):
    ierr, ratea = psspy.brndat(from_bus, to_bus, ckt, "RATEA")
    ierr2, rateb = psspy.brndat(from_bus, to_bus, ckt, "RATEB")
    ierr3, ratec = psspy.brndat(from_bus, to_bus, ckt, "RATEC")

```

```

ierr4, p = psspy.brnmsc(from_bus, to_bus, ckt, "P")
ierr5, q = psspy.brnmsc(from_bus, to_bus, ckt, "Q")
ierr6, pct = psspy.brnmsc(from_bus, to_bus, ckt, "PCTMVA")
if any(i != 0 for i in [ierr, ierr2, ierr3, ierr4, ierr5, ierr6]):
    raise RuntimeError(
        f"Could not read branch data for {from_bus}-{to_bus}-{ckt}, "
        f"ierrs={[ierr, ierr2, ierr3, ierr4, ierr5, ierr6]}"
    )
mva = math.sqrt(p ** 2 + q ** 2)
return {
    "ratea": ratea,
    "rateb": rateb,
    "ratec": ratec,
    "p": p,
    "q": q,
    "mva": mva,
    "pct": pct,
}

def read_case_snapshot(casefile, label):
    open_and_solve(casefile, label)
    bus_data = {}
    for bus in BUSES_TO_CHECK:
        bus_data[bus] = {
            "vpu": get_bus_voltage(bus)
        }
    branch_data = {}
    for br in BRANCHES_TO_CHECK:

```

```

    key = (br["from_bus"], br["to_bus"], br["ckt"])

    branch_data[key] = get_branch_data(br["from_bus"], br["to_bus"], br["ckt"])

    return bus_data, branch_data

# =====

# 5) READ BOTH CASES

# =====

benchmark_buses, benchmark_branches = read_case_snapshot(BENCHMARK_CASE, "BENCHMARK")
test_buses, test_branches = read_case_snapshot(TEST_CASE, "TEST")

# =====

# 6) VALIDATE BUSES

# =====

print("\n" + "=" * 80)

print("BUS VOLTAGE VALIDATION AGAINST 1% THRESHOLD")

print("=" * 80)

bus_pass_all = True

print(f"{'Bus':<10} {'Benchmark PU':>15} {'Test PU':>15} {'Error %':>12} {'Status':>10}")

for bus in BUSES_TO_CHECK:

    ref_v = benchmark_buses[bus]["vpu"]

    tst_v = test_buses[bus]["vpu"]

    err = pct_error(ref_v, tst_v)

    status = "PASS" if err <= THRESHOLD_PERCENT else "FAIL"

    if status == "FAIL":

        bus_pass_all = False

        print(f"{'bus':<10} {'ref_v':>15.6f} {'tst_v':>15.6f} {'err':>12.4f} {'status':>10}")

# =====

# 7) VALIDATE BRANCHES

```

```

# =====

print("\n" + "=" * 80)

print("BRANCH VALIDATION AGAINST 1% THRESHOLD")

print("=" * 80)

branch_pass_all = True

for br in BRANCHES_TO_CHECK:

    key = (br["from_bus"], br["to_bus"], br["ckt"])

    ref = benchmark_branches[key]

    tst = test_branches[key]

    err_mva = pct_error(ref["mva"], tst["mva"])

    err_pct = pct_error(ref["pct"], tst["pct"])

    err_ratea = pct_error(ref["ratea"], tst["ratea"])


# For DLR studies:

# - MVA should usually remain close if dispatch/topology unchanged

# - Loading % and RATEA are expected to change after DLR

# So benchmark the right things depending on your goal.

print(f"\nBranch: {br['name']} ({br['from_bus']}->{br['to_bus']}, ckt {br['ckt']})")

print(f" Benchmark RATEA = {ref['ratea']:.6f}")

print(f" Test RATEA    = {tst['ratea']:.6f}")

print(f" RATEA Error %  = {err_ratea:.4f}")

print(f" Benchmark MVA   = {ref['mva']:.6f}")

print(f" Test MVA       = {tst['mva']:.6f}")

print(f" MVA Error %     = {err_mva:.4f}")

print(f" Benchmark %Load = {ref['pct']:.6f}")

print(f" Test %Load      = {tst['pct']:.6f}")

print(f" %Load Error %    = {err_pct:.4f}")

```

```

# Use MVA as the real steady-state benchmark check

status_mva = "PASS" if err_mva <= THRESHOLD_PERCENT else "FAIL"

print(f" MVA 1% Check  = {status_mva}")

if status_mva == "FAIL":

    branch_pass_all = False

# =====

# 8) FINAL DECISION

# =====

print("\n" + "=" * 80)

print("FINAL VALIDATION SUMMARY")

print("=" * 80)


print(f"Bus voltage 1% validation : {'PASS' if bus_pass_all else 'FAIL'}")
print(f"Branch MVA 1% validation  : {'PASS' if branch_pass_all else 'FAIL'}")

overall = bus_pass_all and branch_pass_all

print(f"Overall benchmark result  : {'PASS' if overall else 'FAIL'}")

```


References

- [1] Alberta Electric System Operator (AESO), "Section 501.10 – Transmission Loss Factors," 2024. [Online]. Available: <https://www.aeso.ca/rules-standards-and-tariff/iso-rules/section-501-10-transmission-loss-factors/>
- [2] IEEE Standard for Calculating the Current-Temperature Relationship of Bare Overhead Conductors, IEEE Std 738-2012. [Online]. Available: <https://ieeexplore.ieee.org/document/6189510>
- [3] P. Kundur, *Power System Stability and Control*. New York, NY, USA: McGraw-Hill, 1994. [Online]. Available: <https://www.amazon.com/Power-System-Stability-Control-Kundur/dp/007035958X>
- [4] Arduino, "Arduino UNO Rev3," [Online]. Available: <https://docs.arduino.cc/hardware/uno-rev3/>
- [5] Analog Devices, "TMP36 Temperature Sensor Datasheet," [Online]. Available: https://www.analog.com/media/en/technical-documentation/data-sheets/tmp35_36_37.pdf
- [6] Adafruit, "Anemometer Sensor – Product 1733," [Online]. Available: <https://www.adafruit.com/product/1733>
- [7] LM1602C LCD Datasheet, [Online]. Available: <https://www.sparkfun.com/datasheets/LCD/LM16XX.pdf>
- [8] LDR Light Sensor Datasheet, [Online]. Available: https://www.electronics-tutorials.ws/io/io_3.html